

# PGR210– Lec 3

## Kristiania University of Applied Sciences

By Huamin Ren

[Huamin.ren@kristiania.no](mailto:Huamin.ren@kristiania.no)

# Outline

- 1. A brief overflight of hyperspace
- 2. Word order and grammar
- 3. Looking at text
- 4. Text pre-processing

# 1. A brief overflight of hyperspace

- Imagine how a single typo or variation in the statement would trip up our bot!

When we use character sequence matches to *measure distance* between natural language phrases, we'll often get it wrong.

Good

Bad

OK

Bar

Distance metrics designed for numerical sequences and vectors are useful for a few NLP applications, like spelling correctors and recognizing proper nouns. So we use these distance metrics when they make sense.

But for NLP applications where we are more interested in the meaning of the natural language than its spelling, there are better approaches.

# 1. A brief overflight of hyperspace

- Can vector representations of natural language words and text together with some distance metrics be a solution to those NLP applications?

The symbols have consistent meaning and aren't changed with every key click to try to confuse us.

This tedious counting of characters and words is just the sort of thing a computer can do without thinking. And surprisingly, it's nearly enough to make the machine appear to understand our language.

# 1. A brief overflight of hyperspace

- Discussion: what information has been lost?

If we assign the words to bins and store them away as bit vectors like a coin or token sorter directing different kinds of tokens to one side or the other in a cascade of decisions that piles them in bins at the bottom, can human understand a document in this way?

**Without any sequence or order associated with those words?**

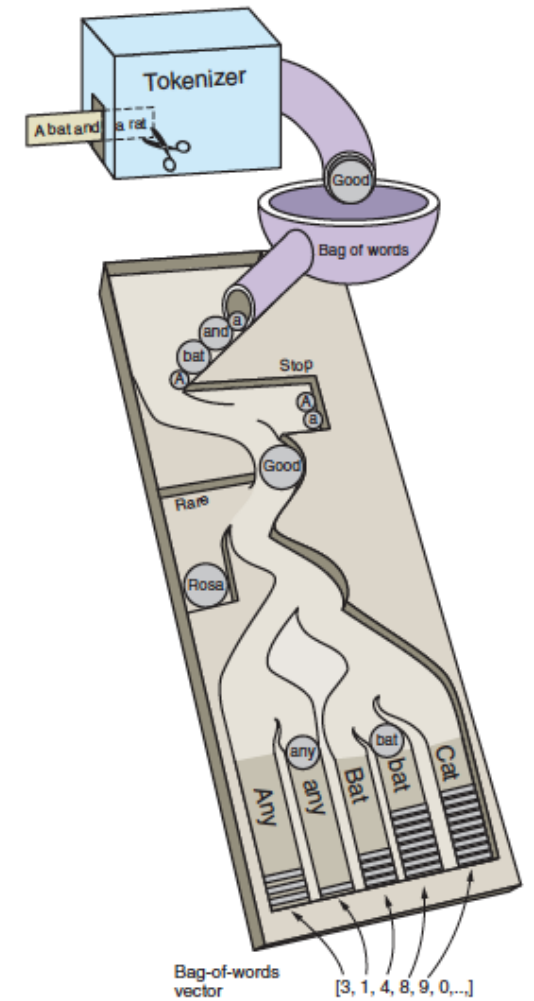


Figure 1.2 Token sorting tray

# Outline

- 1. A brief overflight of hyperspace
- **2. *Word order and grammar***
- 3. Looking at text
- 4. Text pre-processing

## 2. Word order and grammar

- The order of words matters

Those rules that govern word order in a sequence of words (like a sentence) are called the grammar of a language.

- *Fortunately*, in most short phrases and even many complete sentences, this word vector approximation works OK

```
['Good morning Rosa!',  
 'Good Rosa! morning',  
 'morning Good Rosa!',  
 'morning Rosa! Good',  
 'Rosa! Good morning',  
 'Rosa! morning Good']
```

# Outline

- 1. A brief overflight of hyperspace
- 2. Word order and grammar
- **3. *Looking at text***
- 4. Text pre-processing

# 3. Supplemented material – looking at text

- 3.1 Markup
- 3.2 Low-level formatting issues
- 3.3 Tokenization: What is a word?
- 3.4 Sentences

## 3.1 Markup

- Text will usually come in either a raw format, or marked up in some way
- Markup: putting codes of some sort into a markup computer file

Markup are not actually part of the text in the file, but explain something of the structure or formatting of that text.

## 3.2 Low-level formatting issues

Often one needs a filter to **remove** junk content before any further processing begins.

- Junk formatting/content
  - Document headers and separators, type-setter codes, tables and diagrams, garbled data in the computer file
  - If the data comes from OCR (Optical Character Recognition), the OCR process may have introduced problems such as headers, footers and floating material (tables, figures, and footnotes) breaking up the paragraphs of the text
- Uppercase and lowercase
  - Discussion: how to manage this?

## 3.2 Low-level formatting issues

A heuristic approach, all converted to lowercase letters?

Winnie the Pooh?

Ignore the problem?



## 3.3 Tokenization: What is a word?

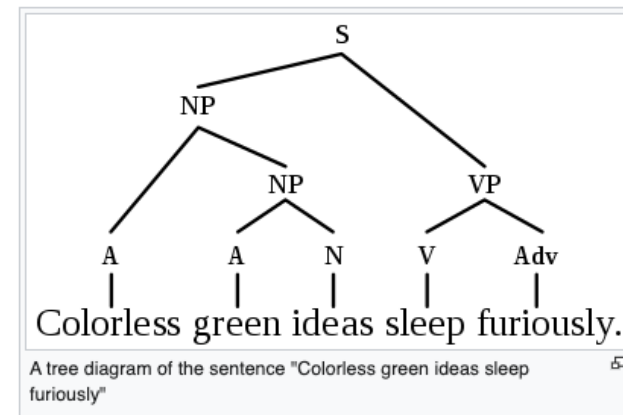
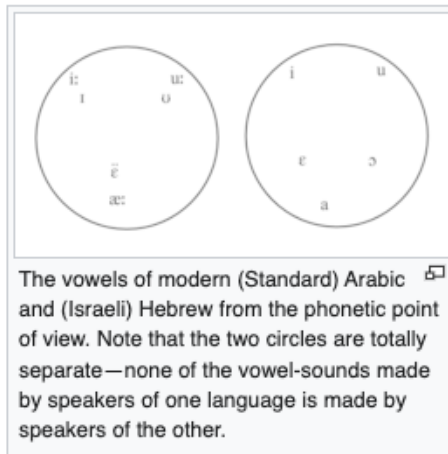
- The most popular tokenization techniques include sentence and word tokenization, which are used to break down a text document (or corpus) into sentences and each sentence into words

Normally, an early step of processing is to divide the input text into units called tokens where each is either a word or something else like a number tokens word or a punctuation mark.

- What counts as a word?

## 3.3 Tokenization: What is a word?

- Linguists end up suggesting that there are words at various levels, such as phonological words versus syntactic words, which need not all be the same.



## 3.3 Tokenization: What is a word?

- Kucera and Francis (1967) suggested:

a graphic word be defined as “a string of contiguous alphanumeric characters with space on either side; may include hyphens (-) and apostrophes (‘), but no other punctuation marks.”

\$22.50

Micro\$oft

: -)

*I'll*

*isn't*

## 3.3 Tokenization: What is a word?

- English language: an estimated 13 million tokens
- Norsk?
- [https://bora.uib.no/bora-xmlui/bitstream/handle/1956/20906/drthesis\\_BjarteJohansen\\_2019.pdf?sequence=1&isAllowed=y](https://bora.uib.no/bora-xmlui/bitstream/handle/1956/20906/drthesis_BjarteJohansen_2019.pdf?sequence=1&isAllowed=y)
- [https://www.duo.uio.no/bitstream/handle/10852/59276/11/Teaching\\_NLTK\\_Norwegian.pdf](https://www.duo.uio.no/bitstream/handle/10852/59276/11/Teaching_NLTK_Norwegian.pdf)

## 3.4 Lexical units & lexicon

- ***Lexical units***: represented by morphemes, the smallest meaningful and syntactically correct unit of a language. Words are inherently a subset of these morphemes.
- ***lexicon*** is a complete vocabulary of these lexical units.
- A rich lexical corpus and database called WordNet, which has an exhaustive list of different lexical entities that are grouped into synsets based on semantic similarity (e.g., synonyms).

## 3.4 Sentences

- What is a sentence?

Something ending with a '.', '?' or '!' ."

"You remind me," she remarked, "of your mother."

- There has been increasing research recently on more principled methods of sentence boundary detection.
- In linguistics classes, and when doing traditional computational linguistics exercises, sentences are generally short. This is at least in part because many of the parsing tools that have traditionally been used have a runtime exponential in the sentence length, and therefore become impractical for sentences over twelve or so words. It is therefore important to realize that typical sentences in many text genres *are rather long*.

# Outline

- 1. A brief overflight of hyperspace
- 2. Word order and grammar
- 3. Looking at text
- **4. *Text pre-processing***

## 4.1 Removing HTML tags

- Unstructured text contains a lot of noise, especially if you use techniques like web scraping or screen scraping to retrieve data from web pages, blogs, and online repositories.
- Hence, HTML tags, Javascript, and Iframe tags typically don't add much value to understanding the text
- Beautiful Soup: <https://beautiful-soup-4.readthedocs.io/en/latest/>

- [Beautiful Soup](#) is a Python library for pulling data out of HTML and XML files. It works with your favorite parser to provide idiomatic ways of navigating, searching, and modifying the parse tree. It commonly saves programmers hours or days of work.

## 4.2 Tokenization

- NLP software typically analyzes text by breaking it up into sentences and words (tokens). Hence, any NLP pipeline has to start with a reliable system to split the text into sentences (sentence segmentation) and further split a sentence into words (word tokenization).
- As a simple rule, sentence segmentation is done by breaking up text into sentences at the appearance of full stops and question marks.
- Similar to sentence tokenization, to tokenize a sentence into words, we can start with a simple rule to split text into words based on the presence of punctuation marks.

## 4.3 Removing unnecessary tokens and stopwords

- Stopwords are words that have little or no significance, and are usually removed from text when processing
  - Stopwords usually occur most frequently if you aggregate a corpus of text based on singular tokens and checked their frequencies. Words like “a,” “the,” “and,” and so on are stopwords
  - There is no universal or exhaustive list of stopwords and often each domain or language has its own set of stopwords
- 
- P3.2